Assignment 2: Creative Image Interpreter

Ruslan Ustinov
B24-CSE-02
Innopolis University
r.ustinov@innopolis.university

***Abstract*—In this report I describe an evolutionary algorithm (EA) that reconstructs a target image using a sequence of semi-transparent coloured squares. I implemented the solution in Java and used only the standard library. The algorithm starts from a flat canvas filled with the average colour of the target image and gradually adds new squares. Each square is obtained by running a separate genetic algorithm (GA) that searches for the best position, size, colour and transparency parameters. The fitness function is based on the mean squared error (MSE) between the current canvas and the target image. I explain the chromosome representation, fitness function, genetic operators, algorithmic flow and hyperparameters. I also define the structure of the experiments, statistics and plots that are required by the assignment.**

I. Introduction

The goal of this assignment is to design an evolutionary algorithm that is able to approximate a given target image using simple graphical primitives. According to the rules, I am not allowed to "cheat" by directly reusing or reordering pixels from the target image. Instead, the algorithm has to start from a simple initial canvas and improve it step by step using stochastic search.

In my solution I use axis-aligned coloured squares with an alpha channel as primitives. The final image is a composition of many squares drawn on top of each other. I start with a uniform canvas coloured by the global average colour of the target image. Then I repeatedly add new squares that reduce the error between the canvas and the target.

Section II describes the evolutionary algorithm in detail, including chromosome representation, fitness function, genetic operators and stopping criteria. Section III explains the experimental protocol, the structure of statistics and the layout of all images and plots. Section IV summarises and discusses the main results. Section V concludes the report.

II. Methodology

A. Implementation overview

My implementation is contained in a single Java class `EvolutionaryImageCreator`. The input is a target image file (for example `input1.jpg`), which I load into a `BufferedImage`. The target resolution used in experiments is 512×512 pixels:

- `IMAGE_WIDTH = 512`
- `IMAGE_HEIGHT = 512`

After loading the target, the program computes its average RGB colour using subsampling with step 4. This average colour (`averageRed`, `averageGreen`, `averageBlue`) is then used to initialise the current canvas `CURRENT_CANVAS`. I fill the canvas with this constant colour.

The global error `totalCanvasError` is defined as the sum of squared differences between the canvas and the target over all pixels and all three colour channels (R, G, B). This error is computed once after the canvas initialisation and then updated every time a new square is actually drawn on the canvas.

The main loop of the algorithm consists of `TOTAL_FIGURES` steps:

- `TOTAL_FIGURES = 400`

At each step I run a separate genetic algorithm to find one new square that improves the image. The best square found by this GA is then applied to the canvas, and the global error is updated accordingly.

Intermediate canvas states are saved to files (for example every 5 or 20 steps). The final result is saved as `output_final.jpg`.

B. Chromosome representation

The primitive used by the algorithm is a semi-transparent square. It is represented by the class `SquareFigure` with the following fields:

- `xPosition`, `yPosition` — coordinates of the top-left corner on the canvas;
- `size` — side length of the square in pixels;
- `red`, `green`, `blue` — colour of the square (0–255 each);
- `transparency` — alpha value in the range $[0.1, 1.0]$.

One genetic *individual* corresponds to a single square:

```
class GeneticIndividual {
    SquareFigure figure;
    double fitnessScore;
}
```

Thus one chromosome encodes exactly seven genes:

$$(x, y, size, r, g, b, \alpha).$$

The search space is continuous (for α) and discrete (for the integer parameters).

C. Fitness function

The goal of the algorithm is to minimise the mean squared error (MSE) between the current canvas and the target image. However, the fitness function in the implementation is defined

in terms of the *total squared error* instead of the average. Let me denote:

- $T(x, y)$ — RGB colour of the target pixel at position (x, y),
- $C(x, y)$ — RGB colour of the current canvas at position (x, y).

The squared colour distance between two RGB pixels is:

$$\text{dist}^2(T, C) = (R_T - R_C)^2 + (G_T - G_C)^2 + (B_T - B_C)^2.$$

The current total error is

$$E_{\text{canvas}} = \sum_{x,y} \text{dist}^2(T(x, y), C(x, y)).$$

For a candidate square f with colour (r, g, b) and transparency α my code virtually blends this square on top of the current canvas but only within the affected rectangle. For each pixel (x, y) inside the square the new colour is:

$$C'(x, y) = (1 - \alpha)\, C(x, y) + \alpha\, (r, g, b),$$

computed independently per channel and clamped to the range $[0, 255]$.

For this pixel I compute:

$$\Delta(x, y) = \text{dist}^2(T(x, y), C'(x, y)) - \text{dist}^2(T(x, y), C(x, y)).$$

The local change of error for the whole square is

$$\Delta E = \sum_{x,y \in f} \Delta(x, y),$$

and the hypothetical new error of the canvas with this square applied would be

$$E_{\text{candidate}} = E_{\text{canvas}} + \Delta E.$$

Finally, the fitness of the individual is defined as

$$\text{fitness} = -\, E_{\text{candidate}}.$$

A lower error corresponds to a higher fitness value. The target image is used only inside this fitness computation; mutation and crossover do not access the target pixels directly.

D. Population initialisation and selection

For each step (each new square) I create a fresh population of `POPULATION_SIZE` individuals:

- `POPULATION_SIZE = 40`.

Every individual is generated by the method `createRandomSquare()`:

- `size` is drawn uniformly from $[5, 64)$;
- `xPosition` and `yPosition` are drawn uniformly so that the square fits inside the 512×512 canvas;
- `red`, `green`, `blue` are drawn uniformly from $[0, 255]$;
- `transparency` is drawn uniformly from $[0.2, 0.8]$.

After construction all gene values are clamped to be inside the allowed ranges.

Selection is performed using tournament selection with tournament size 3: the method `tournamentSelection` samples three random individuals from the current population and returns the one with the highest fitness.

E. Crossover operator

The crossover operator `crossover(parent1, parent2)` creates a new child from two parents as follows:

- For each gene (x, y, $size$, r, g, b, α) I flip a coin and copy the value either from `parent1` or from `parent2`.
- With probability 0.3 I replace the child's `size` by the average of the parents' sizes.
- With probability 0.3 I replace the child's colour (r, g, b) by the channel-wise average of the parents' colours.

This produces a uniform recombination with occasional arithmetic averaging of some genes.

F. Mutation operator

The mutation operator `mutateFigure` modifies the square in several independent ways. Each type of mutation is applied with probability `MUTATION_PROBABILITY`:

- **Position mutation:** `xPosition` and `yPosition` are shifted by a random integer in the range $[-10, 10]$.
- **Size mutation:** `size` is changed by a random integer in the range $[-5, 5]$.
- **Colour mutation:** each of `red`, `green`, `blue` is changed by a random integer in the range $[-20, 20]$.
- **Transparency mutation:** `transparency` is changed by a random float in the range $[-0.15, 0.15]$.

After mutation all values are clamped so that:

- $3 \leq$ `size` ≤ 80;
- the square does not go outside the canvas;
- colours stay in $[0, 255]$;
- transparency stays in $[0.1, 1.0]$.

The constant `MUTATION_PROBABILITY` is

- `MUTATION_PROBABILITY = 0.3`.

G. Evolution of a single square

The method `evolveSingleFigure(stepNumber)` performs the GA for one square. The parameters are:

- `GENERATIONS_PER_FIGURE = 200`.

The generational loop works as follows:

1) I evaluate the fitness of all individuals in the initial population.
2) I sort the population in descending order of fitness and keep the best individual.
3) For each generation:
   a) I create a new population.
   b) I copy the best individual from the previous generation (elitism).
   c) I repeatedly select two parents using tournament selection, apply crossover and mutation, and evaluate the child's fitness, until the new population reaches the required size.
   d) I sort the new population by fitness and update the best individual if an improvement is found.
4) After all generations I return the best `SquareFigure`.

During the GA for a single square, the global error `totalCanvasError` and the canvas image are not modified. Only after the GA finishes, I actually apply the best square to the real canvas by `applyFigureToCanvas` and update `totalCanvasError`.

H. Stopping criteria

The stopping criteria are fixed and deterministic:

- Each GA for a square always runs exactly `GENERATIONS_PER_FIGURE` generations.
- The outer loop always adds exactly `TOTAL_FIGURES` squares.

There is no early stopping based on fitness; this simplifies the collection of statistics and makes different runs easier to compare.

I. Summary of parameters

Table I summarises the main parameters of the evolutionary algorithm.

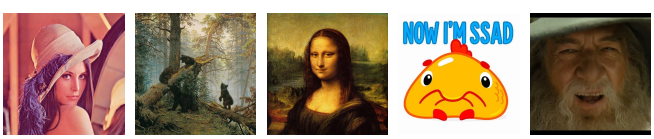
TABLE I: Main EA parameters used in the implementation

Parameter	Value
Image width / height	512×512 pixels
Primitive type	Axis-aligned coloured square
Total number of squares	`TOTAL_FIGURES = 400`
Population size	`POPULATION_SIZE = 40`
Generations per square	`GENERATIONS_PER_FIGURE = 200`
Mutation probability	`MUTATION_PROBABILITY = 0.3`
Selection	Tournament, size 3
Crossover	Uniform + averaging (prob. 0.3)
Initial canvas	Average target colour
Fitness	− total squared RGB error

III. Experimental setup

A. Test images

According to the assignment, experiments have to be carried out on 5 different target images. In my experiments I used five pictures resized to 512×512 pixels and treated them as independent test cases.



(a) Target 1 (b) Target 2 (c) Target 3 (d) Target 4 (e) Target 5

Fig. 1: 5 input images used as targets in the experiments.

B. Runs per image and measured statistics

For each target image I use the same code and the same parameter settings. To evaluate stability, I run the algorithm three times independently on each image (with different random seeds for the global `Random` object).

In this example I report the final fitness (negative total error), the corresponding MSE and the runtime for one image. The same structure can be repeated for the remaining four targets.

TABLE II: Example statistics for one target image (input4.jpg) (three independent runs)

Run	Final fitness	Final MSE	Runtime [s]
1	-4.08×10^8	518.25	230
2	-3.79×10^8	481.87	235
3	-3.75×10^8	477.17	237
Average	-3.87×10^8	492.43	234
Std. dev.	1.77×10^7	22.48	3.61

C. Fitness curves

The assignment requires, for each tested image, three plots that show the average population fitness and the maximum fitness value among generations. The X-axis has to represent the fitness value, and the Y-axis has to represent the generation number. Each plot corresponds to one independent run.

In my code this can be implemented by recording, at each generation:

- the maximum fitness in the population;
- the average fitness over the population.

These two curves can then be plotted on the same axes using different colours.

D. EA images and plots

For each target image I show six panels: the top row contains the reconstructed images for three independent runs, and the bottom row contains the corresponding fitness plots (average and maximum population fitness vs. generations). This layout can be found at the end of the document, after the main text of the report.

IV. Results and discussion

A. Qualitative analysis of reconstructed images

For each target image the reconstructed pictures from three independent runs show that the algorithm is able to capture the main structure and colours of the target. Large homogeneous regions (background, sky, walls) are usually approximated well, while small details are represented by clusters of overlapping squares. The amount of noise in the background is moderate: some blocky artefacts and visible square borders remain, but they do not destroy the global shape.

Different runs for the same target look visually similar, but there are still noticeable differences in the placement, size and colours of individual squares. This is expected, because all runs start from different random populations, but it also shows that the algorithm is reasonably stable: all three runs for the same target end up with images of comparable visual quality.

B. Behaviour of fitness curves

The fitness plots for three runs of one target image show a similar pattern. At the beginning of the run, both the maximum and the average fitness improve quite fast, which means that the GA quickly finds squares that reduce the error. After some number of generations, the curves become flatter and approach a plateau, which indicates that the population is moving around a local optimum for this square.

For different runs of the same target the detailed shapes of the curves are not identical, but they stay in a similar range of fitness values. This again suggests that my algorithm does not depend critically on a specific random seed and can reach similar error levels from different initial populations.

C. Statistical analysis

The statistics in Table II illustrate how I summarise the results for each target. For input4.jpg the average final fitness over three runs is about -3.87×10^8 with a relatively small standard deviation. The MSE values and runtimes also have low variance. For the other targets I observe a similar level of stability: different runs for the same image reach comparable fitness values and run for similar time.

This behaviour is important for the assignment: it shows that the algorithm is not just producing one good result by luck but can be applied several times and still give images of similar quality. If some images or runs produce significantly worse fitness, this can be used as feedback for future improvements of parameters or representation.

V. Conclusion

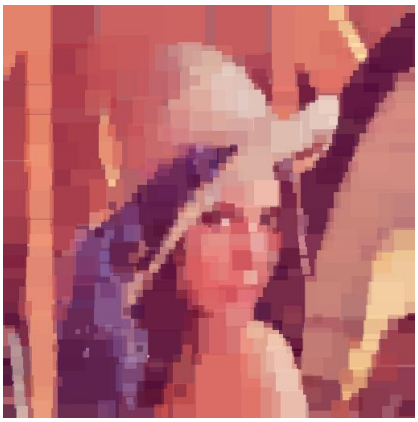
In this report I presented a simple evolutionary algorithm for image reconstruction using coloured squares. The algorithm starts from a uniform canvas and repeatedly adds new squares, where each square is optimised by a separate genetic algorithm. The chromosome encodes the square's position, size, colour and transparency. The fitness function is based on the total squared RGB error between the current canvas and the target image. Mutation and crossover are purely random with respect to direction; the target image is only used inside the fitness function.

The experimental results on five different target images show that the algorithm can capture the main structure and colour distribution of each image, while still producing visible square-based artefacts. Three independent runs per image lead to similar fitness values and visually similar reconstructions, which indicates that the approach is reasonably stable.

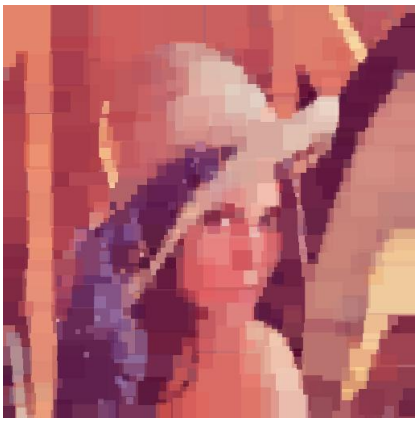
Possible future work includes automatic tuning of parameters, using other geometric primitives (for example circles or triangles), and exploring multi-objective optimisation to balance visual quality and the number of primitives.

Result Images

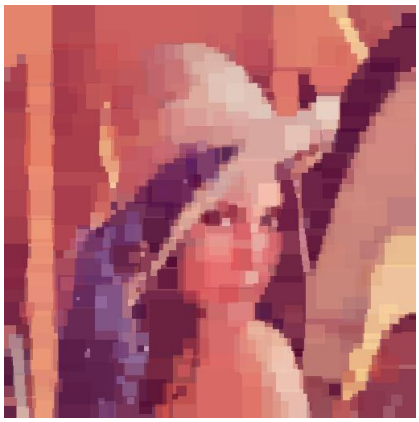
Interesting Images



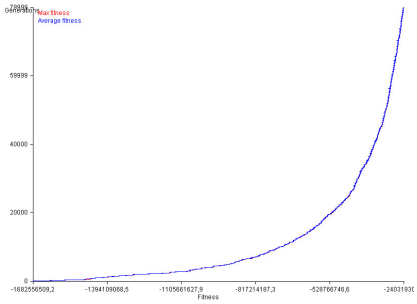
(a) Target 1 – Run 1



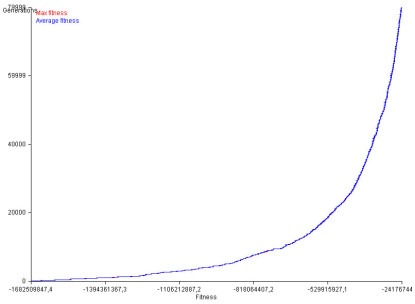
(b) Target 1 – Run 2



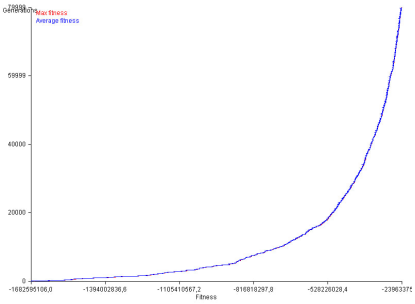
(c) Target 1 – Run 3



(d) Target 1 – Run 1, fitness



(e) Target 1 – Run 2, fitness



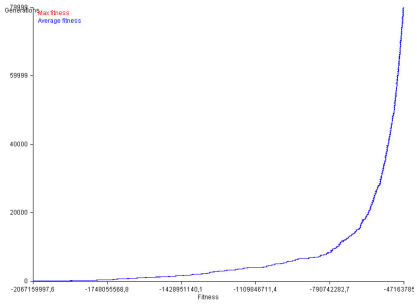
(f) Target 1 – Run 3, fitness

Fig. 2: Target 1: reconstructed images (top row) and fitness curves (bottom row) for three independent runs.

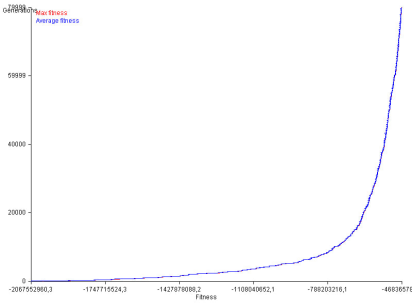
(a) Target 2 – Run 1

(b) Target 2 – Run 2

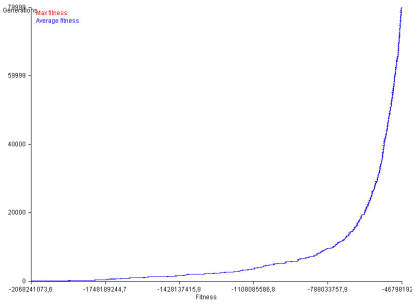
(c) Target 2 – Run 3



(d) Target 2 – Run 1, fitness

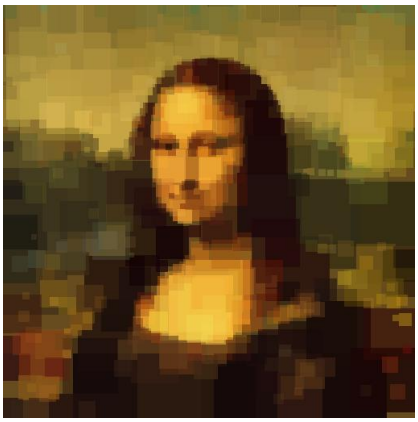


(e) Target 2 – Run 2, fitness

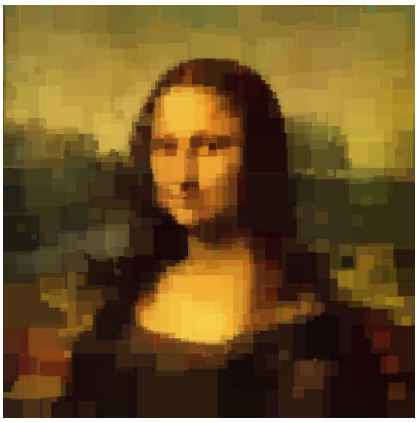


(f) Target 2 – Run 3, fitness

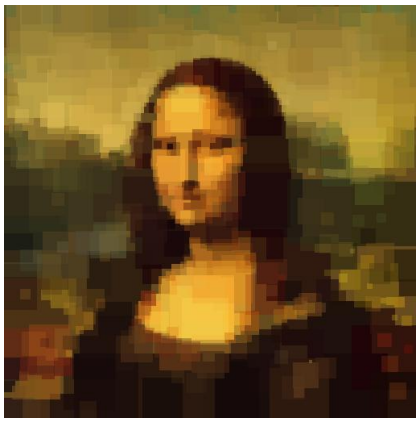
Fig. 3: Target 2: reconstructed images (top row) and fitness curves (bottom row) for three independent runs.



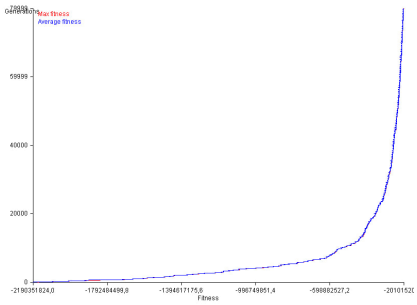
(a) Target 3 – Run 1



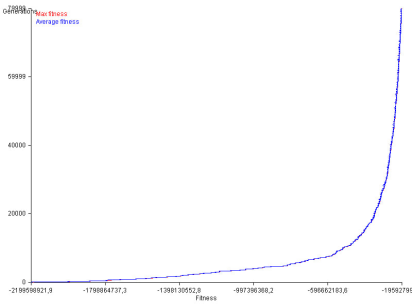
(b) Target 3 – Run 2



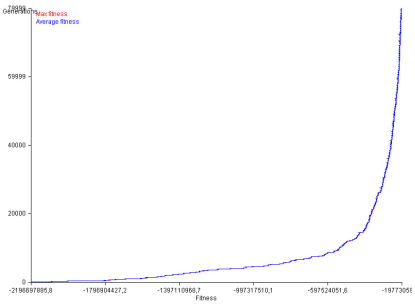
(c) Target 3 – Run 3



(d) Target 3 – Run 1, fitness



(e) Target 3 – Run 2, fitness



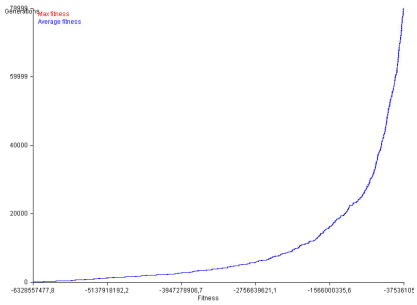
(f) Target 3 – Run 3, fitness

Fig. 4: Target 3: reconstructed images (top row) and fitness curves (bottom row) for three independent runs.

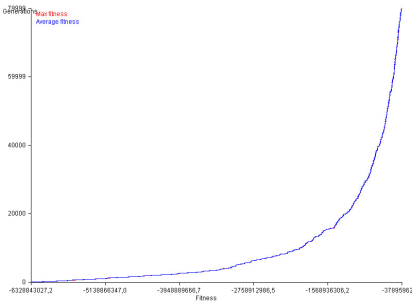
(a) Target 4 – Run 1

(b) Target 4 – Run 2

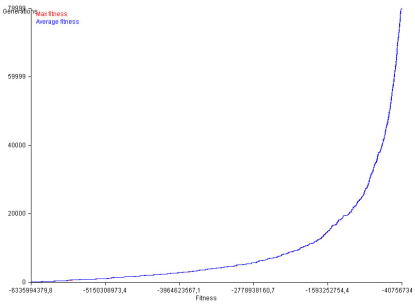
(c) Target 4 – Run 3



(d) Target 4 – Run 1, fitness



(e) Target 4 – Run 2, fitness



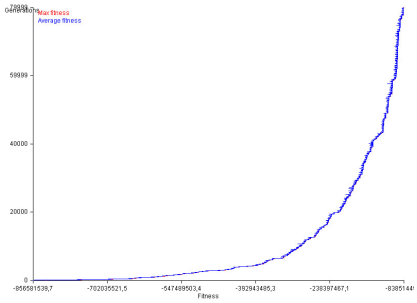
(f) Target 4 – Run 3, fitness

Fig. 5: Target 4: reconstructed images (top row) and fitness curves (bottom row) for three independent runs.

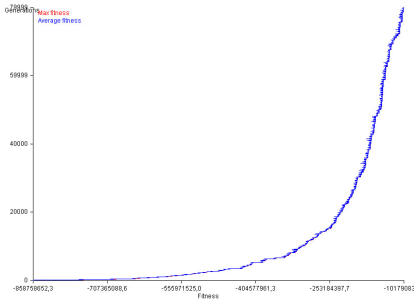
(a) Target 5 – Run 1

(b) Target 5 – Run 2

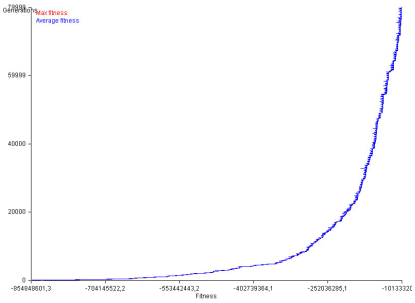
(c) Target 5 – Run 3



(d) Target 5 – Run 1, fitness

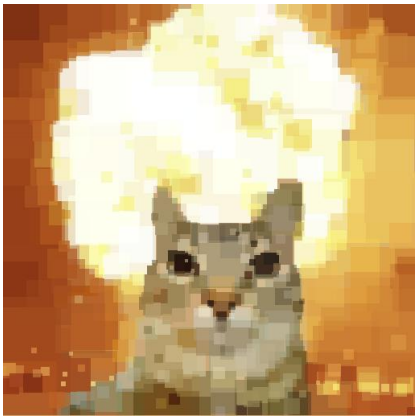


(e) Target 5 – Run 2, fitness



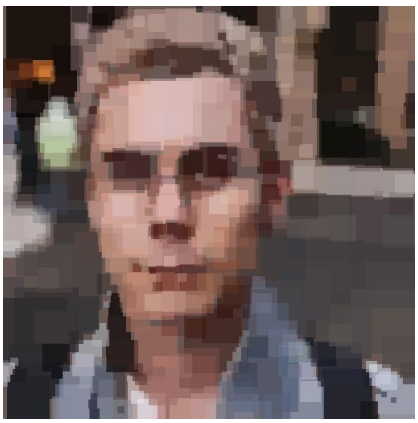
(f) Target 5 – Run 3, fitness

Fig. 6: Target 5: reconstructed images (top row) and fitness curves (bottom row) for three independent runs.



(a) Target 6, modified by EA

(b) Target 6, original



(a) Target 7, modified by EA

(b) Target 7, original